# AIML PROJECT REPORT

## Food Image Classifier with Calorie Estimation

**COURSE :** Artificial Intelligence / Machine Learning
**NAME :** SURYA SNATO SINHA
**REGISTRATION NUMBER :** 25BCE10164

## PROJECT OVERVIEW

This project presents a Convolutional Neural Network (CNN)-based system that classifies food images and estimates their calorie content. The model is trained on a dataset of labeled food images and is capable of identifying different food items from user-uploaded images. After classification, the system maps the predicted food item to a predefined nutritional database to estimate calorie values. The project demonstrates the practical application of deep learning in solving real-world problems related to health and nutrition.

## Objectives

To build an image classification model using CNN
To classify food items from images
To estimate calorie content of detected food
To develop a simple user interface for interaction

## Methodology

### 1. Data Collection

A dataset containing images of food items such as apple, banana, rice, egg, and chicken was used. The dataset is organized into labeled folders for supervised learning.

## 2. Data Preprocessing

Images resized to 128x128 pixels
Pixel values normalized (0–1 range)
Dataset split into training and validation sets

## 3. Model Architecture

A Convolutional Neural Network (CNN) was used consisting of:

Convolutional layers (feature extraction)
MaxPooling layers (downsampling)
Fully connected layers (classification)
Softmax activation for multi-class output

## 4. Training

Optimizer: Adam
Loss Function: Sparse Categorical Crossentropy
Epochs: 5–10
Model trained on labeled dataset

## 5. Prediction and Calorie Estimation

The trained model predicts the food category
A dictionary maps each food item to its approximate calorie value

## 6. User Interface

A simple web interface was developed using Streamlit where users can:

Upload food images
View predicted label
See estimated calories and confidence score

## Results

The model achieved satisfactory accuracy on the validation dataset. It successfully classifies common food items and provides approximate calorie estimates. The system works efficiently for basic food categories.

## SCREENSHOTS

```
import streamlit as st
from predict import predict_image
from PIL import Image
import tempfile

st.title("🍔 Food Image Classifier with Calories")

uploaded_file = st.file_uploader("Upload a food image", type=["jpg", "png", "jpe

if uploaded_file is not None:
    # Show image
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Image", use_column_width=True)

    # Save temp file
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        tmp.write(uploaded_file.read())
        file_path = tmp.name

    # Prediction
    label, confidence, calorie = predict_image(file_path)

    st.success(f"Detected Food: {label}")
    st.info(f"Confidence: {confidence*100:.2f}%")
    st.warning(f"Estimated Calories: {calorie} kcal per serving")
```

```
train_data = tf.keras.preprocessing.image_dataset_from_directory(
    "dataset",
    image_size=(128, 128),
    batch_size=32,
    validation_split=0.2,
    subset="training",
    seed=123
)

val_data = tf.keras.preprocessing.image_dataset_from_directory(
    "dataset",
    image_size=(128, 128),
    batch_size=32,
    validation_split=0.2,
    subset="validation",
    seed=123
)

class_names = train_data.class_names
print(class_names)


normalization_layer = layers.Rescaling(1./255)
train_data = train_data.map(lambda x, y: (normalization_layer(x), y))
val_data = val_data.map(lambda x, y: (normalization_layer(x), y))


model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    layers.MaxPooling2D(),
```

```
normalization_layer = layers.Rescaling(1./255)
train_data = train_data.map(lambda x, y: (normalization_layer(x), y))
val_data = val_data.map(lambda x, y: (normalization_layer(x), y))


model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    layers.MaxPooling2D(),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)


history = model.fit(train_data, validation_data=val_data, epochs=5)
```

```
train_data = tf.keras.preprocessing.image_dataset_from_directory(
    "dataset",
    image_size=(128, 128),
    batch_size=32,
    validation_split=0.2,
    subset="training",
    seed=123
)

val_data = tf.keras.preprocessing.image_dataset_from_directory(
    "dataset",
    image_size=(128, 128),
    batch_size=32,
    validation_split=0.2,
    subset="validation",
    seed=123
)

class_names = train_data.class_names
print(class_names)


normalization_layer = layers.Rescaling(1./255)
train_data = train_data.map(lambda x, y: (normalization_layer(x), y))
val_data = val_data.map(lambda x, y: (normalization_layer(x), y))

# CNN Model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    layers.MaxPooling2D(),
```


```
normalization_layer = layers.Rescaling(1./255)
train_data = train_data.map(lambda x, y: (normalization_layer(x), y))
val_data = val_data.map(lambda x, y: (normalization_layer(x), y))

# CNN Model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    layers.MaxPooling2D(),
```


```
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)


history = model.fit(train_data, validation_data=val_data, epochs=5)
```